ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES
INGENIERÍA EN INFORMÁTICA · Sede Alameda
Asignatura: Capstone (PTY4614)

INFORME DE AVANCEPROYECTO APT — FASE 2“Terreno Conectado, Decisiones en Tiempo Real”Datos al día, donde no hay señal.

Integrantes
Daniel Ávila — RUT: 21.256.327-7
Raúl González — RUT: 21.357.918-5
Docente: Carlos Herrera

Santiago de Chile · octubre de 2026

Índice

El índice con números de página se genera automáticamente en el documento Word (.docx).

Abstract

“Terreno Conectado, Decisiones en Tiempo Real” is an offline-first, multi-tenant SaaS platform designed for the Chilean mining and construction industries. Remote work sites usually operate with unstable or non-existent network coverage, which forces field teams to register inspections, logbooks, and operational data on paper or in spreadsheets. This delayed and error-prone capture makes management decide on outdated information, duplicates reporting effort, and lets safety risks remain unnoticed for longer than they should.

During Phase 2 the project moved from a conceptual proposal to a working, validated software system. The team built a complete **walking skeleton** and seven functional modules: authentication and multi-tenancy, offline capture, a synchronization engine, the inspection domain, a management dashboard, platform administration, and a dynamic-forms subsystem. The system is implemented as a TypeScript monorepo (React 18 + Vite PWA, NestJS, PostgreSQL 16 with Row-Level Security, and a Dexie/IndexedDB outbox), and it is verified by an automated test suite of **394 tests** (unit, integration against a real PostgreSQL, and end-to-end), a requirement-to-test coverage matrix at **94.9 %**, and a seven-scenario chaos protocol that reported zero data loss, zero duplicates, and zero cross-tenant leaks.

This Phase 2 progress report explains the adjustments made to the original proposal, the methodology that guided the work, and the evidence produced for the ten quality indicators of the graduate profile. Its main finding is that building the validation layer early was decisive: the manually executed validation checklist detected real defects —including a seeded-data issue that affected a new user's first-run experience— that no code review had identified before.

Keywords: offline-first, multi-tenant SaaS, Spec Driven Development, data synchronization, software validation.

Resumen

“Terreno Conectado, Decisiones en Tiempo Real” es una plataforma SaaS *offline-first* y multiempresa orientada a la minería y la construcción en Chile. Las faenas y obras remotas operan sin cobertura de red estable, lo que obliga a capturar inspecciones, bitácoras y datos operacionales en papel o planillas. Esa captura tardía hace que la gerencia decida con información desactualizada, duplica el trabajo de reporte y retrasa la detección de riesgos de seguridad.

Durante la Fase 2 el proyecto pasó de una propuesta conceptual a un **sistema de software funcional y validado**. Se construyó un **walking skeleton** completo y siete módulos: autenticación y multi-tenancy, captura offline, motor de sincronización, dominio de inspecciones, dashboard de gerencia, administración de plataforma y formularios dinámicos. La solución se implementó como un monorepo TypeScript (React 18 + Vite PWA, NestJS, PostgreSQL 16 con Row-Level Security y un *outbox* en Dexie/IndexedDB) y se verifica con **394 pruebas** (unitarias, de integración contra un PostgreSQL real y end-to-end), una matriz de cobertura requisito→prueba con **94,9 %** de filas en verde y un protocolo de siete escenarios de caos que reportó cero pérdidas, cero duplicados y cero fugas entre empresas.

Este informe de avance describe los ajustes a la propuesta original, la metodología de trabajo y las evidencias que dan cuenta de los diez indicadores de calidad del perfil de egreso. Su hallazgo principal es que construir la capa de validación a tiempo fue decisivo: la validación manual ejecutada detectó defectos reales —incluido un problema de datos sembrados que afectaba la primera experiencia de un usuario nuevo— que ninguna revisión de código había identificado.

Palabras clave: offline-first, SaaS multi-tenant, Spec Driven Development, sincronización de datos, validación de software.

1. Introducción y contexto del Proyecto APT1.1 El proyecto

“Terreno Conectado, Decisiones en Tiempo Real” es una plataforma de software como servicio (SaaS) que permite a trabajadores de terreno capturar datos operacionales **sin conexión a internet** y sincronizarlos automáticamente con la nube cuando la señal se recupera. Su propósito central es acortar la distancia entre lo que ocurre en una faena minera o una obra de construcción y el momento en que esa información llega a quienes deben decidir sobre ella.

El proyecto nace de un problema operativo concreto y vigente en dos de los sectores productivos más relevantes de Chile: la **minería** y la **construcción**. En ambos, gran parte de la actividad ocurre en terrenos remotos con cobertura intermitente o inexistente, lo que históricamente ha obligado a registrar inspecciones y bitácoras en papel o planillas para transcribirlas después. El resultado es información desactualizada para la gerencia, trabajo administrativo duplicado y riesgos de seguridad detectados más tarde de lo deseable.

1.2 Relevancia y aporte de valor

El aporte de valor del proyecto —de carácter simulado en el marco de la asignatura, pero fundamentado en una problemática real— es **reducir la brecha de tiempo entre la captura del dato en terreno y su disponibilidad para la toma de decisiones**. Para lograrlo, la propuesta no trata la aplicación de terreno como un formulario que envía datos “si hay red”, sino que invierte el supuesto: **el terreno nunca depende de la red**. La captura es siempre local y persistente, y la sincronización es un proceso posterior, automático y auditable.

Desde la perspectiva de la Ingeniería en Informática, el proyecto sitúa al equipo frente a un desafío característico de la transformación digital de industrias tradicionales: sectores económicamente relevantes, con procesos de datos poco maduros y con condiciones de operación exigentes (conectividad limitada, múltiples empresas cliente, integridad de datos crítica). Diseñar y validar una solución que funcione en esas condiciones es una competencia demandada en el mercado laboral local.

1.3 Relación con las competencias del perfil de egreso

El proyecto ejercita de forma equilibrada las cuatro competencias del perfil de egreso asociadas a esta asignatura:

Competencia del perfil de egreso	Cómo se aborda en el proyecto
Realizar pruebas de certificación de productos y procesos con buenas prácticas de la industria	Plan de pruebas, matriz de cobertura, protocolo de caos y validación manual del checklist de aceptación.
Gestionar proyectos informáticos, ofreciendo alternativas para la toma de decisiones	Constitución del proyecto, specs, planificación por módulos, tablero, hitos y monitoreo del plan de trabajo.
Construir modelos de datos escalables en el tiempo	Modelo conceptual y físico de 18 tablas, aislamiento multi-tenant con RLS y *outbox* local.
Desarrollar una solución de software con técnicas que sistematicen el desarrollo y mantenimiento	Metodología Spec Driven Development, monorepo, CI/CD, revisiones por pares y despliegue.

1.4 Intereses profesionales del equipo

Ambos integrantes comparten el interés por el desarrollo de software orientado a resolver problemas operativos reales de industrias tradicionales chilenas, con foco en backend, infraestructura en la nube y modelamiento de datos. **Daniel Ávila** profundiza en el modelamiento de datos y la lógica de negocio; **Raúl González**, en la infraestructura, la plataforma como servicio y la validación. La sección 10 retoma esta dimensión desde una reflexión individual, en inglés, al cierre de la Fase 2.

2. Objetivos2.1 Objetivo general

> *Diseñar y validar una propuesta de plataforma de recolección de datos corporativos para la industria minera y de construcción, capaz de operar sin conexión constante a internet y de sincronizar la información de forma automática, con el fin de reducir la brecha de tiempo entre el terreno y la toma de decisiones gerenciales.*

2.2 Objetivos específicos

- 1. **Levantar y fundamentar** el problema de la desconexión en faenas remotas y su impacto operacional.
- 2. **Proponer una arquitectura** de captura de datos que funcione sin depender de conexión constante.
- 3. **Definir un modelo multiempresa (SaaS)** que permita ofrecer el servicio a múltiples empresas cliente manteniendo la separación y seguridad de sus datos.
- 4. **Planificar y ejecutar las etapas de trabajo** necesarias para validar la propuesta dentro de los plazos de la asignatura.

2.3 Alcance del avance de Fase 2

Los objetivos de la definición original **no fueron reformulados**. Lo que cambió fue su **nivel de concreción**: donde la Fase 1 hablaba de una “propuesta” y de una “maqueta demostrativa”, la Fase 2 entregó una **implementación funcional y validada** de la arquitectura. Este cambio de alcance es el ajuste más importante de la fase y se justifica en la sección 6.4.

3. Metodología de trabajo3.1 Spec Driven Development (SDD) sobre iteraciones ágiles

El equipo adoptó **Spec Driven Development (SDD)** como metodología central, apoyada en iteraciones cortas inspiradas en marcos ágiles. El principio rector es que **todo artefacto de código nace de un requisito trazable**: ninguna tarea se ejecuta sin un requisito que la justifique.

El proyecto se rige por una **constitución** de siete artículos innegociables (Art. I *offline-first* real; Art. III idempotencia y LWW; Art. IV aislamiento por motor con RLS; Art. V trazabilidad; Art. VI presupuesto; entre otros). Cualquier cambio que la contradiga exige una **enmienda numerada** aprobada por el equipo antes de implementarse.

3.2 El ciclo por módulo

Cada módulo sigue siempre el mismo ciclo:

```
spec → plan → tasks → código → tests → evidencia
```

- **spec**: qué debe hacer el módulo y qué requisitos (FR/NFR) lo definen.
- **plan**: diseño técnico derivado del spec.
- **tasks**: descomposición en tareas, cada una con su requisito asociado.
- **código y tests**: implementación con pruebas que verifican **comportamiento observable**.
- **evidencia**: registro del resultado que alimenta el plan de pruebas del proyecto.

Los *specs* se escriben en **español**; el código, los tests, los commits y el CI, en **inglés**. Un spec congelado solo se modifica mediante una **enmienda numerada** tramitada por un *pull request*.

3.3 Organización del equipo, roles y zonas de trabajo

Aunque el proyecto es grupal y las decisiones se toman en conjunto, el trabajo se dividió en **zonas de propiedad** para evitar conflictos y reducir el riesgo de pisarse:

Integrante	Rol	Zona principal de trabajo
Daniel Ávila	Responsable de datos y lógica de negocio	`frontend/src/lib`, `frontend/src/screens`, `frontend/test`, `backend/src/dashboard`, `backend/src/sync`, specs 002/004/005, modelo de datos
Raúl González	Responsable de infraestructura, plataforma y validación	`specs/001`, `infra/`, `plans` y `tasks` de 001, `package.json` raíz, `frontend/scripts`, validación y despliegue

Los archivos **compartidos** (`test-plan.md` y `README.md`) se editan por filas o secciones propias, y todo cambio de código pasa por **revisión recíproca obligatoria**: cada *pull request* requiere **una aprobación del otro integrante**, con un *time-box* de **24 horas**.

3.4 Estándares de ingeniería y aseguramiento de la calidad

El equipo fijó estándares mínimos verificables de forma automática:

- **Gate obligatorio en cada PR:** `lint` + `typecheck` + `test`. Un PR no se aprueba si el gate no está en verde.
- **Integración continua (CI):** tres *jobs* en cada push y PR — (1) lint + typecheck + build + verificación de la PWA, (2) pruebas de integración con **Testcontainers** sobre un PostgreSQL 16 real, y (3) escaneo de secretos con **gitleaks**.
- **Trazabilidad:** cada tarea referencia un requisito (FR/NFR) y cada commit referencia su *issue* (`refs #N`).
- **Decisiones documentadas:** las decisiones arquitectónicas se registran como **ADR** (por ejemplo, ADR-001 monorepo TypeScript, ADR-002 arquitectura de despliegue, ADR-003 motor de formularios).

3.5 Pertinencia de la metodología

La metodología es pertinente por tres razones concretas. **Primero**, el dominio del problema exige corrección técnica (no perder datos, no duplicar, no filtrar entre empresas): el enfoque *spec-first* con pruebas de comportamiento hace verificable cada promesa. **Segundo**, el equipo es pequeño y con tiempo acotado, por lo que separar “qué se

promete” (spec) de “cómo se implementa” (plan) permitió paralelizar el trabajo por zonas sin romper la integración. **Tercero**, la Fase 2 requería evidencias de avance continuas y no un producto terminado al final, y el ciclo por módulo produce exactamente eso: cada módulo cierra con su spec, su código, sus pruebas y su evidencia.

4. Desarrollo de ingeniería — avance de la Fase 24.1 Arquitectura de la solución

La solución es un **monorepo TypeScript** de cuatro *workspaces* (`shared`, `frontend`, `backend`, `infra`) y tres capas. El terreno nunca depende de la red: la captura ocurre en el dispositivo y la sincronización es posterior.

Capa	Tecnología	Decisión / motivo
Lenguaje	**TypeScript** full-stack	un solo lenguaje en el equipo (ADR-001)
Frontend	React 18 + Vite 6, PWA	instalable en Android de gama media
Almacenamiento local	**Dexie / IndexedDB**	*outbox* transaccional en el dispositivo
Service worker	**Workbox**	precache del shell (funciona sin red tras la primera carga)
Backend	**NestJS** (monolito modular)	estructura modular y tipos compartidos
Base de datos	**PostgreSQL 16 + RLS**	aislamiento multi-tenant a nivel de motor
Identidad	JWT propio (access 15 min + refresh)	claims de tenant, costo $0, sin *lock-in*
Identidad de registro	**UUIDv7** generado en cliente	creación offline sin depender del servidor
Sincronización	Cola propia + ingesta idempotente por lotes	el corazón del proyecto, no se delega
Fotos	Compresión en cliente (canvas)	costo $0; evolución a S3 documentada
Infra	1 VPS + Docker Compose + Caddy	TLS automático, ~US$5/mes
CI/CD	GitHub Actions	3 *jobs* por push y PR

4.2 Módulo 001 — Autenticación, sesión y tenancy

Se implementó autenticación JWT propia con *access* de 15 minutos y *refresh* de larga duración, **multi-tenancy** con `tenant_id` en cada tabla de dominio y políticas **Row-Level Security** aplicadas en PostgreSQL (`SET LOCAL app.tenant_id`). La sesión se restaura desde la caché local cuando no hay red, de modo que un usuario ya autenticado puede seguir capturando offline. La validación manual detectó aquí una discrepancia relevante (hallazgo **N-01**): el checklist referenciaba `user.rol` mientras la API expone `user.role`; se documentó como corrección de nomenclatura en la documentación de validación.

4.3 Módulo 002 — Captura offline

El frontend es una **PWA instalable** (manifest + service worker Workbox) que funciona sin red tras la primera carga. La captura **persiste en el dispositivo**: cada ítem de inspección se autoguarda en Dexie/IndexedDB, el borrador se restaura al reabrir, y hallazgos y bitácora quedan encolados en un **outbox real** con IDs **UUIDv7** de cliente y orden de dependencia. La fotografía del hallazgo se **comprime en el dispositivo** antes de encolarse (canvas ≤ 1280 px, calidad 0.7). Ocho pantallas cubren el flujo: Login, Captura, Bitácora, Cola, Gerencia, Conflictos, Plantillas e Importar, con el ciclo `pending → syncing → synced` visible para el usuario.

4.4 Módulo 003 — Motor de sincronización

La cola local la consume un **worker de sincronización**: al recuperar señal (evento `online`) o justo tras encolar, dispara el *flush* contra `POST /api/v1/sync/batch` con **concurrencia 1** y **backoff exponencial 1 s → 5 min**. La ingesta en el servidor es **idempotente** (upsert por UUIDv7 del cliente) y los conflictos se resuelven con **Last-Write-Wins determinista**, dejando **ambas versiones auditadas** en `CONFLICT_RECORD` (nunca hay sobrescritura silenciosa). Cada fila pasa `pending → syncing → synced`, o `failed` con contador de intentos y `lastError`. La latencia captura→disponibilidad se mide y se expone al dashboard.

4.5 Módulo 004 — Dominio de inspecciones

Se modeló el dominio de inspecciones: plantillas versionadas e inmutables, ítems con respuesta híbrida (`value_json` más `ATTACHMENT` para fotos), hallazgos con severidad, sitios/faenas y bitácora. Las reglas de integridad se validan en el cliente y en el servidor con un **catálogo Zod en `shared/`**, lo que garantiza que frontend y backend validen exactamente las mismas reglas.

4.6 Módulo 005 — Reportes y gerencia

El **dashboard del supervisor** opera con datos reales del tenant: métricas de inspecciones por estado, hallazgos por severidad, **latencia media y p95** captura→disponibilidad (`synced_at − captured_at`), destacado de hallazgos `alta/crítica`, filtros por período, faena y severidad, auto-refresh, y **exportación CSV**. Los conflictos se listan con ambas versiones (ganador/perdedor). Todo se sirve desde `GET /api/v1/sites`, `GET /api/v1/dashboard/summary` y `GET /api/v1/dashboard/findings`.

4.7 Módulo 006 — Administración de plataforma

Módulo de operación de la plataforma (administración de tenants, usuarios y control), apoyado en la bitácora de auditoría (`audit_log`) y en el registro de operaciones (`log_entry`). El diseño mantiene una separación estricta entre la administración de la plataforma y los datos de dominio de cada empresa cliente.

4.8 Módulo 007 — Formularios dinámicos

El módulo de formularios dinámicos avanza desde un **motor de render* data-driven***: plantillas versionadas e inmutables, catálogo Zod compartido, respuesta híbrida,*** *importador `.xlsx` con wizard de revisión humana* ***(la importación no es "mágica": el usuario revisa antes de confirmar),*** *asignación por faena y rol****,*** *delta sync con caché offline* ***y*** *export rellenado con hoja de Evidencias (dispositivo, dashboard y lote). De las tareas `TSK-FORM-001→010` están completas; queda pendiente `TSK-FORM-011` (pruebas,* issue *#44) y la versión 2 (*issue* #45).

4.9 Modelo de datos

El modelo de datos quedó consolidado en **18 tablas** de dominio, agrupadas en cuatro dominios: *tenancy y usuarios*, *plantillas e inspecciones*, *sincronización y auditoría*, y *cliente local* (IndexedDB). Se documenta en `data-model.md` con diagrama entidad-relación, reglas de integridad y control de cambios. Su implementación se realiza mediante **migraciones versionadas** con `node-pg-migrate` (**12 migraciones**, desde `1730000000000_init-tenancy`), e incluye las **políticas RLS** que hacen del aislamiento multi-tenant una garantía del motor y no solo de la aplicación. La sección 7.7 y 7.8 detallan esta evidencia.

4.10 Infraestructura, CI/CD y despliegue

La infraestructura local se levanta con **Docker Compose** (PostgreSQL 16 + RLS) y la API se compila a `dist/`. El CI corre tres *jobs* en cada push y PR. Para el despliegue se diseñó y documentó una arquitectura de demo de **bajo costo** en un solo VPS con Docker Compose y **Caddy** (TLS automático) del orden de **US$5/mes**, junto con un plan de despliegue de producción real (**~US$200–600/mes**) que queda documentado en el ADR correspondiente pero **no provisionado**, en coherencia con el Artículo VI (presupuesto). El provisioning efectivo del VPS de demo es la única actividad de infraestructura **no iniciada**, y depende del cierre del HITO 2 y de la entrega de credenciales (ver sección 6.5).

5. Monitoreo del plan de trabajo5.1 Plan de trabajo y estado de avance

La siguiente tabla retoma el plan de trabajo de la Fase 1 y lo actualiza con el **estado de avance** y los **ajustes** de cada actividad, de acuerdo con la Guía 2.4.

Competencia	Actividad	Recursos	Duración	Responsable	Observaciones	Estado	Ajustes
Gestionar proyectos	Investigación y definición del problema	Fuentes bibliográficas, tiempo de equipo	2 sem	Daniel y Raúl	Facilitador: información pública abundante	Completado	—
Gestionar proyectos	Definición de objetivos y metodología	Reuniones, plantilla institucional	1 sem	Daniel y Raúl	Facilitador: guía del docente en taller	Completado	Se adoptó SDD sobre iteraciones ágiles
Construir modelos de datos	Modelamiento conceptual de datos	Diagramas colaborativos	2 sem	Daniel	Obstaculizador: sin datos reales	Completado	Escaló a modelo físico de 18 tablas
Desarrollar solución	Diseño conceptual de la solución	Referencias de la industria	2 sem	Raúl	Facilitador: interés previo en infraestructura	Completado	Se materializó como arquitectura real
Gestionar proyectos	Definición del modelo multiempresa (SaaS)	Investigación de modelos SaaS	1 sem	Daniel y Raúl	Obstaculizador: complejidad del tema	Ajustada	Se resolvió como tenancy con RLS
Desarrollar solución	Construcción de prototipo/maqueta	Herramientas de prototipado, equipos propios	4 sem	Daniel y Raúl	Obstaculizador: tiempo compartido con otras asignaturas	Completado	Ver sección 6.4 (alcance)
Realizar pruebas	Diseño y ejecución de pruebas de validación	Plan de pruebas, casos de prueba	2 sem	Raúl	Facilitador: revisiones periódicas	Completado	Se sumó validación manual y caos
Gestionar proyectos	Elaboración de informes y evidencias	Plantillas institucionales	Transversal	Daniel y Raúl	Obstaculizador: acumulación al cierre	En curso	Avances parciales periódicos
Gestionar proyectos	Preparación de presentaciones	Material de apoyo visual	1 sem/fase	Daniel y Raúl	Facilitador: retroalimentación entre fases	Completado (F1)	—
Desarrollar solución	*Walking skeleton* (TSK-WS-001→012)	Monorepo, CI	4 sem	Daniel y Raúl	Hito habilitante de todo lo demás	Completado	—
Desarrollar solución	Módulos 001–006	Backend, frontend, BD	6 sem	Daniel y Raúl	Por zonas de propiedad	Completado	—

Desarrollar solución	Módulo 007 (formularios dinámicos)	Motor de render, XLSX	4 sem	Daniel y Raúl	TSK-FORM-001→010	En curso	Pendiente TSK-FORM-011 (#44)
Realizar pruebas	Plan de pruebas + harness E2E	Playwright, Testcontainers	2 sem	Raúl	Cierra el *issue* #10	Completado	—
Realizar pruebas	Validación V1 (checklist + caos)	Build PWA, API, checklist	1 sem	Raúl	Cierra el *issue* #130	Completado	Detectó F-01, F-02, N-01
Desarrollar solución	Despliegue en VPS de demo	VPS, Docker Compose, Caddy	1 sem	Raúl	Depende de HITO 2 + credenciales	No iniciado	Se ejecutará al cierre del HITO 2

5.2 Estado por objetivo específico

Objetivo específico	**Estado**	**Evidencia principal**
1. Levantar y fundamentar el problema	**Cumplido**	Informe Fase 1; contexto del proyecto (§1)
2. Proponer una arquitectura de captura sin conexión	**Cumplido y superado**	Arquitectura implementada (§4.1–4.4); PWA + outbox + worker
3. Definir un modelo multiempresa seguro	**Cumplido**	Tenancy con RLS; 18 tablas; matriz cross-tenant en caos
4. Planificar y ejecutar las etapas de validación	**En curso**	Plan de pruebas; 394 pruebas; validación V1; cierre de HITO 2

6. Ajustes a la propuesta6.1 Factores facilitadores

- **Interés previo y complementario del equipo** en backend, infraestructura y modelamiento de datos, lo que permitió repartir el trabajo por fortalezas.
- **Fuentes públicas abundantes** sobre la brecha de conectividad en minería y construcción, que permitieron fundamentar el problema sin acceso privilegiado a una faena real.
- **Retroalimentación docente al cierre de cada fase**, que permitió corregir el rumbo de forma temprana.
- **Estandarización temprana** (constitución, specs, ADR y CI) antes de escribir código, que redujo retrabajo posterior.
- **Herramientas de costo cero** (Docker local, GitHub Actions, Testcontainers) que hicieron viable la validación automatizada dentro del presupuesto.

6.2 Dificultades y cómo se abordaron

Dificultad	Acción tomada
Sin acceso a una faena real para observar el problema en terreno	Se usaron fuentes documentales de la industria y **escenarios simulados de desconexión** reproducibles.
Carga académica simultánea de otras asignaturas	Planificación por iteraciones, zonas de propiedad y reuniones breves de seguimiento.
Curva de aprendizaje en multi-tenancy SaaS	División por fortalezas y semanas iniciales de investigación dirigida antes de diseñar.
La validación manual no reproducía el entorno real (el bloque offline solo es válido contra el build de producción PWA)	Se estableció que las pruebas offline deben correr contra el **build PWA** con `vite preview` y modo offline real, no contra el *dev server*.
Dependencia de Docker/Testcontainers para las pruebas de integración	Se documentó la dependencia y se dejó el *harness* E2E como orquestador reproducible.

6.3 Retroalimentación recibida

El equipo se apoyó en **tres fuentes de retroalimentación** además de la heteroevaluación docente del cierre de fase:

- 1. **Retroalimentación docente (Fase 1).** El instrumento de evaluación formativa de la Fase 1 destacó la necesidad de mantener el foco en la **factibilidad** dentro del tiempo de la asignatura y de justificar el **aporte de valor** con evidencia. El equipo respondió acotando el alcance a una plataforma de captura y sincronización, con un despliegue de demo de bajo costo.
- 2. **Retroalimentación entre pares (*code review*). La revisión recíproca obligatoria (una aprobación, *time-box 24 h*) actuó como control de calidad permanente: cada *pull request** se aprobó únicamente con el **gate** (lint, typecheck, tests) en verde, y varios cambios se reformularon antes de integrarse.
- 3. **Retroalimentación del propio producto (validación).** La ejecución del checklist de validación produjo hallazgos concretos que obligaron a ajustar el sistema (sección 6.4), lo que constituye la evidencia más fuerte del indicador de mejora basada en resultados.

6.4 Actividades ajustadas o eliminadas

- **“Construcción de un prototipo o maqueta” → sistema funcional validado (ajuste principal).** En la Fase 1 se planificó una *maqueta* de las pantallas principales. Con la metodología y la infraestructura ya disponibles, el equipo decidió construir un **sistema real** con *walking skeleton* y siete módulos, porque permitía evidenciar de forma mucho más sólida los indicadores 3.x y 4.x, y porque validar el comportamiento *offline-first* sobre una maqueta conceptual habría sido imposible. El riesgo se controló manteniendo el despliegue de producción real fuera de alcance (solo demo).
- **“Modelo de negocio SaaS” → tenancy con RLS.** El objetivo 3 se reinterpretó: en lugar de elaborar un *modelo de negocio* formal (fuera del alcance ingenieril de la asignatura), se materializó el requisito esencial —**servir a varias empresas con datos aislados**— como **multi-tenancy con Row-Level Security**, verificable y demostrable.
- **Incorporación de la validación manual y del caos.** No previstas en detalle en la Fase 1, se agregaron porque la corrida del checklist de aceptación (issue #130) y los siete escenarios de caos revelaron defectos que las pruebas

automatizadas no cubrían.

6.5 Actividades no iniciadas o retrasadas

- **Despliegue en el VPS de demo (no iniciado).** Depende de cerrar el **HITO 2** del plan de cierre (F2–F3) y de la entrega de credenciales del VPS. La arquitectura y el plan de despliegue ya están documentados; la ejecución es el paso siguiente.
- **Módulo 007 — tarea `TSK-FORM-011` (retrasada,* issue *#44).** Corresponde a la batería de pruebas del motor de formularios. Es la pieza que falta para el cierre del HITO 2.
- **Cierre del HITO 2 (en curso).** Falta la matriz de aceptación por rol ≥ 90 % de la zona de Daniel.

  Estrategia de recuperación: mantener el orden de cierre definido en el plan (cerrar pruebas del 007 → HITO 2 → provisioning del VPS → demo), con avances parciales periódicos para no acumular trabajo al final.

7. Evidencias de avance7.1 Estrategia de evidencias y trazabilidad

Todas las evidencias del proyecto son **verificables y reproducibles**: viven en el repositorio, se generan con comandos documentados y se referencian desde el plan de pruebas. La trazabilidad es completa: cada requisito (FR/NFR) apunta a una prueba, cada tarea apunta a un requisito y cada commit apunta a su *issue*. Las subsecciones siguientes organizan la evidencia según los **diez indicadores de calidad** del perfil de egreso que exige la pauta.

7.2 Indicador 1.1 — Diseña pruebas de validación

Evidencia: `test-plan.md` (plan de pruebas de validación) y `docs/checklist-validacion-manual.md`.

El plan de pruebas define un **principio rector** (“cada promesa del spec tiene una prueba que la verifica”), seis **niveles de prueba** (unit, integración API, E2E, caos offline, adversarial cross-model, rendimiento y manual documentado), la **matriz de cobertura requisito→prueba**, el **protocolo de caos** de siete escenarios y los criterios de aceptación. El checklist de validación manual especifica el recorrido crítico ⭐ (H-01…H-10) y la **matriz rol × pantalla** (7 pantallas × 4 roles). El diseño distingue explícitamente pruebas **automatizadas** y **manuales**, y establece que las pruebas validan **comportamiento observable** (¿se pierde el dato?, ¿se duplica?, ¿cruza tenants?).

7.3 Indicador 1.2 — Aplica pruebas de validación

Evidencia: ejecución del conjunto de pruebas y de la validación V1.

- **394 pruebas** en total: **77** en `shared`, **112** en `frontend` y **188** en `backend` (unitarias y de integración contra PostgreSQL 16 real vía Testcontainers), más **17** pruebas E2E/caos con Playwright en perfil móvil.
- **Matriz de cobertura (`test-plan.md` §3): 39 filas**, **37 en verde (94,9 %)**, 1 parcial y 1 pendiente.
- **Protocolo de caos (7 escenarios):** corte de red, replay, LWW concurrente, reinicio hostil, cola masiva, token vencido y cross-tenant, con **0 pérdidas, 0 duplicados y 0 fugas cross-tenant**.
- **Validación manual (checklist, issue #130):** recorrido crítico **H-01…H-10 en 10/10**, matriz rol × pantalla **OK**, y casos de API (§2.4-B/§2.5/§3) aprobados. Resultados en `docs/informe-pruebas-validacion.md`.

7.4 Indicador 1.3 — Mejoras en base a los resultados

Evidencia: hallazgos y correcciones registradas en `docs/informe-pruebas-validacion.md`.

La validación **no fue un trámite**: encontró defectos reales que ninguna revisión de código había detectado.

Hallazgo	Descripción	Mejora aplicada
F-01	El seed sembraba registros de demostración en la cola, afectando la primera experiencia de un usuario nuevo.	Se eliminó la siembra de datos de cola: `COLA_INICIAL` quedó vacía y `seedDemoDataIfNeeded` pasó a ser *no-op*, conservando solo cuentas demo y plantilla local. **PR #133 mergeado.**
F-02	El bloque offline del checklist solo es válido contra el **build de producción PWA** (con SW), no contra el *dev server*.	Se documentó el entorno de validación correcto (build + `vite preview` + offline real).
N-01	El checklist referenciaba `user.rol` mientras la API expone `user.role`.	Se registró la discrepancia de nomenclatura para corregir la documentación.

Además, la validación dejó **un spec de Playwright conservado en el repositorio** (`frontend/e2e/specs/checklist-smoke.spec.ts`, 5 pruebas) para que el recorrido crítico quede como **prueba de regresión** permanente. El cierre de esta mejora se consolidó en el **PR #132** (informe y spec).

7.5 Indicador 2.1 — Planifica proyectos informáticos

Evidencia: `constitution.md`, `specs/`, `plans/`, `tasks/`, `plans/cierre-f2-f3/plan.md`, `test-plan.md` y el plan de trabajo de este informe.

La planificación se expresa en una **pirámide de artefactos** que va de lo estable a lo ejecutable: constitución (principios innegociables) → spec maestro (visión, módulos, 52 requisitos FR y 12 NFR) → specs por módulo → planes técnicos → tareas. La Fase 2 se organizó por **módulos** con criterios de cierre, y el tramo final por **hitos** (HITO

1, HITO 2, HITO 3) con responsables, entregables y dependencias explícitas. El plan de trabajo y la Carta Gantt (sección 5) ofrecen la vista de actividades, recursos, duración y responsables.

7.6 Indicador 2.2 — Controla proyectos informáticos

Evidencia: CI (`.github/workflows/ci.yml`), historial de *pull requests* y tablero del equipo.

El control es **continuo y automático**: tres *jobs* de CI por push y PR, un **gate obligatorio** (lint + typecheck + tests) que condiciona cada aprobación, **revisión recíproca** con *time-box* de 24 horas y **trazabilidad** commit→issue→requisito. El monitoreo del plan de trabajo (sección 5) mantiene el estado por actividad y registra los ajustes, y el plan de cierre F2–F3 define los hitos y sus criterios de aceptación para detectar desviaciones antes del cierre.

7.7 Indicador 3.1 — Diseña modelos de datos

Evidencia: `data-model.md` y `adr/`.

El modelo de datos comprende **18 tablas** de dominio organizadas en cuatro dominios, con **diagrama entidad-relación**, reglas de integridad, consideraciones de **multi-tenancy mediante RLS** y el modelo del **cliente local** (IndexedDB). El diseño es **escalable por construcción**: toda tabla de dominio lleva `tenant_id` y política RLS, la identidad de registro usa **UUIDv7 generado en cliente** (permite crear entidades sin servidor y evita colisiones al sincronizar) y las plantillas se **versionan de forma inmutable** para no romper registros históricos. Las decisiones estructurales están respaldadas por ADR.

7.8 Indicador 3.2 — Implementa modelos de datos

Evidencia: `backend/db/migrations/` y el *outbox* local.

El modelo se implementa con **12 migraciones versionadas** (`node-pg-migrate`), desde `1730000000000_init-tenancy` hasta migraciones posteriores de formularios y ubicación de inspección. Cada migración crea tablas, llaves, índices y **políticas RLS**. En el cliente, el modelo se implementa con **Dexie/IndexedDB**, con la tabla de cola (*outbox*) y los borradores persistidos. Los *tests* de integración verifican el comportamiento del esquema contra un PostgreSQL real, incluidas las **políticas de aislamiento**.

7.9 Indicador 4.1 — Construye una solución de software

Evidencia: el repositorio del proyecto (monorepo con cuatro *workspaces*).

La solución se construye con técnicas que **sistematizan** el desarrollo: SDD, monorepo con contratos compartidos, componentes por módulo, *walking skeleton* como base habilitante y CI desde temprano. El resultado es una base de código coherente: frontend de 8 pantallas, backend NestJS con auth/sync/conflictos/dashboard/plantillas, y `shared/` con tipos y validaciones comunes.

7.10 Indicador 4.2 — Integra los distintos componentes

Evidencia: monorepo, contratos `shared/`, CI de build y pruebas E2E de integración.

La integración se garantiza de varias formas: un **contrato único** de tipos y validaciones en `shared/` (con Zod) consumido por frontend y backend; el **build** y el **typecheck** de los tres *workspaces* en CI; las pruebas de integración que ejercitan la API completa contra PostgreSQL real; y las **pruebas E2E** que levantan el sistema completo (PostgreSQL → migraciones → seed → backend → frontend compilado) y verifican los flujos de extremo a extremo, incluidos los escenarios de caos.

7.11 Indicador 4.3 — Implanta una solución de software

Evidencia: build de producción PWA, `infra/` (Docker Compose + Caddy) y el plan de despliegue.

La solución se **empaqueta e implanta**: la PWA se compila a `dist/` con *service worker* y *precache* (verificado por CI), la base de datos y la API se levantan con **Docker Compose**, y el despliegue de demo se resuelve con **Caddy** para TLS automático en un solo VPS de bajo costo. El despliegue de producción real está **diseñado y documentado** (ADR) pero deliberadamente no provisionado por presupuesto. El **provisioning del VPS de demo** es la última actividad de implantación, planificada para el cierre del HITO 2.

7.12 Resumen de evidencias

Indicador de calidad	Evidencia principal	Ubicación
1.1 Diseña pruebas	Plan de pruebas + checklist	`test-plan.md`, `docs/checklist-validacion-manual.md`
1.2 Aplica pruebas	394 pruebas, matriz 94,9 %, caos, V1	CI, `test-plan.md` §3–§4, `docs/informe-pruebas-validacion.md`
1.3 Mejoras por resultados	Hallazgos F-01/F-02/N-01 y PRs	`docs/informe-pruebas-validacion.md`, PR #132/#133
2.1 Planifica	Constitución, specs, planes, hitos	`constitution.md`, `specs/`, `plans/`, `tasks/`
2.2 Controla	CI, gate, revisiones, trazabilidad	`.github/workflows/ci.yml`, PRs
3.1 Diseña datos	Modelo de 18 tablas + ER + RLS	`data-model.md`, `adr/`
3.2 Implementa datos	12 migraciones + outbox	`backend/db/migrations/`, Dexie
4.1 Construye	Monorepo y módulos	repositorio
4.2 Integra	Contratos `shared/`, E2E, CI build	`shared/`, `frontend/e2e/`
4.3 Implanta	Build PWA, Compose, Caddy, plan deploy	`frontend/dist`, `infra/`, ADR

8. Resultados de validación8.1 Pruebas automatizadas

Workspace	Archivos	Pruebas
`shared`	5	77
`frontend`	14	112
`backend`	17	188
Subtotal (unit + integración)	**36**	**377**
E2E + caos (Playwright)	1 suite	17
Total		**394**

8.2 Matriz de cobertura requisito → prueba

La matriz de `test-plan.md` §3 tiene **39 filas**; a la fecha de este informe **37 están en verde (94,9 %)**, 1 parcial y 1 pendiente. La matriz cubre los dominios de autenticación, captura offline, sincronización, conflictos, dashboard, formularios y las promesas no funcionales (latencia, aislamiento, resiliencia).

8.3 Protocolo de caos

Los siete escenarios de caos se ejecutan contra el sistema completo en perfil móvil y cerraron con **0 pérdidas de datos, 0 duplicados y 0 fugas entre empresas**:

Escenario	Resultado
Corte de red	Sin pérdida; la cola se drena al reconectar
Replay de lote	Idempotente: sin duplicados
LWW concurrente	Conflicto resuelto y auditado (ganador/perdedor)
Reinicio hostil	Estado local consistente tras reinicio
Cola masiva	Drenaje completo con backoff
Token vencido	Renovación/reauth controlada
Cross-tenant	Aislamiento verificado (0 fugas)

8.4 Validación manual (checklist) y hallazgos

El recorrido crítico **H-01…H-10** se validó **10/10** contra el build de producción PWA y la API reales, la **matriz rol × pantalla** (7 pantallas × 4 roles) resultó correcta, y se aprobaron los casos de API previstos. De esta validación surgieron los hallazgos **F-01, F-02 y N-01**, ya descritos en la sección 7.4 y corregidos o documentados. El informe completo de resultados y el spec de regresión quedaron en el repositorio (`docs/informe-pruebas-validacion.md` y `frontend/e2e/specs/checklist-smoke.spec.ts`).

9. Conclusions

Phase 2 confirmed that the decision with the greatest impact on the project was to invest in the validation layer while the system was still being built. The proposal began the semester as a conceptual architecture for offline data capture; it ends this phase as a working, multi-tenant application whose central promises — not losing data, not duplicating it, and not leaking it across companies — are not asserted but verified. A test suite of 394 tests, a requirement-to-test coverage matrix at 94.9 %, and a seven-scenario chaos protocol are the concrete expression of that shift.

The most valuable lesson, however, came from the manually executed validation checklist rather than from the automated suite. Reviewing the application as a real user would —screen by screen and role by role— surfaced defects that no code review had caught, including a seeded-data issue that shaped a new user's very first experience. This finding validated a design principle the team had formalized early: automated tests confirm that the code does what the team told it to do, but only an independent validation pass reveals whether what the team asked for was actually right.

The methodology proved to be the project's backbone. Adopting Spec Driven Development over short iterations allowed a two-person team to parallelize work along clearly owned zones without breaking integration, because the contracts lived in shared specs rather than in individual assumptions. Maintaining a mandatory quality gate on every pull request, together with reciprocal reviews and full commit-to-requirement traceability, kept the codebase coherent as seven modules were added on top of the walking skeleton.

Regarding the specific objectives, the first two are fully met and the second —proposing an architecture that works without a constant connection— was exceeded, since it moved from a proposal to a functioning implementation. The multi-tenant objective is met through row-level security, and the planning objective is progressing, with the remaining work concentrated in closing the acceptance matrix and provisioning the demo environment.

For Phase 3, the priorities are clear and narrow: complete the outstanding form-engine tests, close the acceptance matrix to bring the overall result above the 90 % target, provision the low-cost demo environment, and consolidate the final report. The project arrives at the end of this phase not with a promise but with a system that can be shown, tested, and defended.

10. Individual Reflections10.1 Raúl González — Infrastructure, platform and validation

Working on the infrastructure and validation side of this project changed how I think about software quality. Before this phase, I tended to treat testing as something that happens once the code is finished. Building the test harness first — and then acting on what it found— showed me that validation is not a final checkpoint but a design activity that runs alongside the whole project. Writing the acceptance checklist and executing it manually, screen by screen and role by role, was uncomfortable at first because it forced me to look at the application as a stranger would; but that discomfort is exactly what exposed defects the automated suite could not see.

I also learned that constraints can improve engineering decisions. The rule of keeping every promise tied to a requirement, together with the budget limit that kept the deployment design at one small server, forced the team to justify each component instead of adding technology out of habit. Planning the deployment, the network requirements, and the costs —even when the real environment was not provisioned yet— made the solution feel like an actual product rather than a class exercise.

Professionally, this phase reinforced my interest in infrastructure and platform work, and added a dimension I did not expect: I now see validation and observability as part of the same discipline as building. My goal for the remaining phase is to close the deployment and the acceptance matrix, and to keep proving that a solution designed for the field can be verified with the same rigor as one designed for a data center.

10.2 Daniel Ávila — Data and business logic

This phase taught me how much of a system's correctness is decided in the data model. My work was focused on how information is structured and how it behaves after being captured offline, and I learned that questions which look trivial on paper —what identifies a record, what happens when two versions of it exist, how a template can change without rewriting history— are exactly where the difficult decisions live. Designing identifiers that are created on the client, and making the synchronization idempotent, were decisions that only made sense once I accepted that the network cannot be trusted.

The dynamic forms module pushed me to think about the boundary between flexibility and safety. Making the system data-driven means users can define their own forms, but it also means the validation rules have to be shared by both the client and the server so they can never disagree. Building that single shared contract taught me more about integration than any diagram could.

Personally, this project confirmed the interest that motivated it in the first place: working on the data and business-logic layer of a system that has to be reliable under difficult operating conditions. It also showed me that the most valuable feedback I received was the one that came from using the product, not from reading the code. For the next phase I want to deepen my understanding of distributed consistency and to finish the work on the forms engine with the same standard of evidence the rest of the project has set.

10.3 Group reflection

Looking back at Phase 2, the team agrees that the hardest part was not writing the code but keeping the project honest. It is easy to declare a feature finished; it is much harder to demonstrate that it keeps its promises when the network disappears, when a batch is replayed, or when two companies share the same database. The decision to build the validation layer early turned out to be the most expensive and the most profitable choice of the phase, and the defects it uncovered —especially the seeded-data issue that degraded a new user's first run— justified its cost several times over.

The methodology also shaped the way the team works together. Dividing the project into owned zones, with shared contracts defined in specs and a mandatory review from the other member, let two people move in parallel without stepping on each other. The reciprocal review was not a formality: it repeatedly caught problems before they reached the main branch, and it distributed knowledge of the whole system between both members rather than locking each one into a single area.

The team enters the final phase with a clear and short list of tasks —finish the pending form-engine tests, close the acceptance matrix above the target, provision the demo environment, and consolidate the final report— and with the confidence that the system built during this phase is a real foundation, not a mock-up.

Referencias

- Corporación de Desarrollo Tecnológico (CDT), Cámara Chilena de la Construcción. (s.f.). *Índice de Transformación Digital en la Construcción*. https://www.cdt.cl/
- Deloitte Chile & CESCO. (2026). *Diagnóstico del ecosistema digital de la minería en Chile*.
- Mundo Minería. (2026, 8 de enero). *Inspecciones mineras en terreno sin internet: un desafío operativo creciente*.
- Portal Minero. (2026, 30 de julio). *Minería avanza en digitalización, pero la gestión de datos sigue siendo una brecha*.
- Proyecto “Terreno Conectado”. (2026). *Repositorio del proyecto: constitución, specs, planes y plan de pruebas*. GitHub.

AnexosAnexo A — Trazabilidad requisito → prueba (extracto)

Requisito	Promesa	Nivel de prueba	Estado
FR-012	La foto se comprime en el dispositivo antes de encolar	Unit (frontend)	Verde
FR-013 / FR-015	IDs UUIDv7 de cliente / orden de dependencia	Unit + Integración	Verde
FR-014	El borrador se restaura al reabrir	Unit (frontend)	Verde
FR-016 / FR-022 / FR-026	Reintentos, backoff y estado `failed`	Unit + E2E	Verde
FR-035	Destacado de hallazgos alta/crítica	Integración	Verde
FR-040	Latencia media/p95 captura→disponibilidad	Integración	Verde
NFR-07	Presupuesto/latencia de despliegue	Rendimiento	Verde
NFR-12	Cada promesa del spec tiene una prueba	Transversal	Verde

Anexo B — Resultados de la validación manual (checklist)

Bloque	Casos	Resultado
Recorrido crítico ⭐	H-01 … H-10	10/10 OK
Matriz rol × pantalla (§4)	7 pantallas × 4 roles	OK
Casos de API (§2.4-B, §2.5, §3)	X-05, X-06, PA-05…PA-13, TB-04/05	Aprobados
Hallazgos	F-01, F-02, N-01	Corregidos / documentados

Anexo C — Estructura del repositorio (extracto)

```
terreno-conectado/
├── constitution.md        ← principios innegociables
├── specs/                 ← 000-master … 007
├── plans/ · tasks/        ← por módulo e iteración
├── data-model.md          ← 18 tablas + RLS
├── test-plan.md           ← plan de pruebas (matriz + caos)
├── adr/                   ← decisiones arquitectónicas
├── frontend/              ← PWA React 18 + Dexie + worker
├── backend/               ← API NestJS + migraciones
├── shared/                ← tipos y validaciones comunes
└── infra/                 ← Docker Compose (+ Caddy, deploy)
```

Anexo D — Cómo reproducir la evidencia principal

```
npm install
npm run lint && npm run typecheck
npm test                       # 394 pruebas (unit + integración)
# E2E + caos (requiere Docker):
npx playwright test            # perfil móvil
```

Fin del informe de avance — Fase 2.